

LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Name: Spoorthi Poornachandra

SRN: PES1UG24AM279

Section: E

Date: 08/09/2026

Scenario 10: Student Club Event Ticketing & Budget Portal

1. Objective

The objective of this lab is to evaluate different architectural styles, select the most appropriate architecture for a given system scenario, and create a UML Component Diagram representing the system's components, interfaces, interactions, and dependencies. The lab focuses on understanding how components interact within a system and how architectural decisions impact overall system design, including their implications for security and performance.

2. Scenario Description

Student Club Event Ticketing & Budget Portal

The system under design is a Student Club Event Ticketing & Budget Portal, a platform used by student clubs to manage the full lifecycle of an event. The portal allows a Club Lead to submit event proposals, which are then reviewed and approved through a sequential budget approval workflow involving a Finance Officer, a Faculty Coordinator and a Dean. Once an event is approved, it becomes visible on an event calendar, and attendees can register for tickets through a QR-based check-in system. The portal must therefore store and manage event proposals, approval records, ticket registrations, QR check-in data and scheduling information, while enforcing role-based access for the different user types involved. Because these responsibilities are distinct but closely related and share a common data set, the system requires several cooperating components rather than a single undivided application.

3. Architectural Style Analysis

Architecture	Advantages	Disadvantages	Suitability for Our Scenario
Layered Architecture	Clear separation of concerns; easy to understand and maintain	Can introduce performance overhead; layers can become tightly coupled	Workable, but does not clearly separate the portal's functional modules (proposals, approvals, ticketing, calendar)

Microservices Architecture	Independent deployment and scaling; fault isolation; technology diversity	High operational complexity; network latency; data consistency challenges across services	Overkill for a relatively small, manageable student portal with a single database
Client-Server Architecture	Centralized control; simple deployment; easier data consistency	Single point of failure; scalability bottleneck; limited fault tolerance	Possible as a deployment model, but on its own does not describe how the portal's internal modules are organised
Modular Monolith	Clear modular separation of responsibilities while remaining a single deployable system; simpler development, testing and maintenance	Less independently scalable than microservices; modules still share one runtime and database	Most suitable — matches the portal's closely related but logically separate functional areas

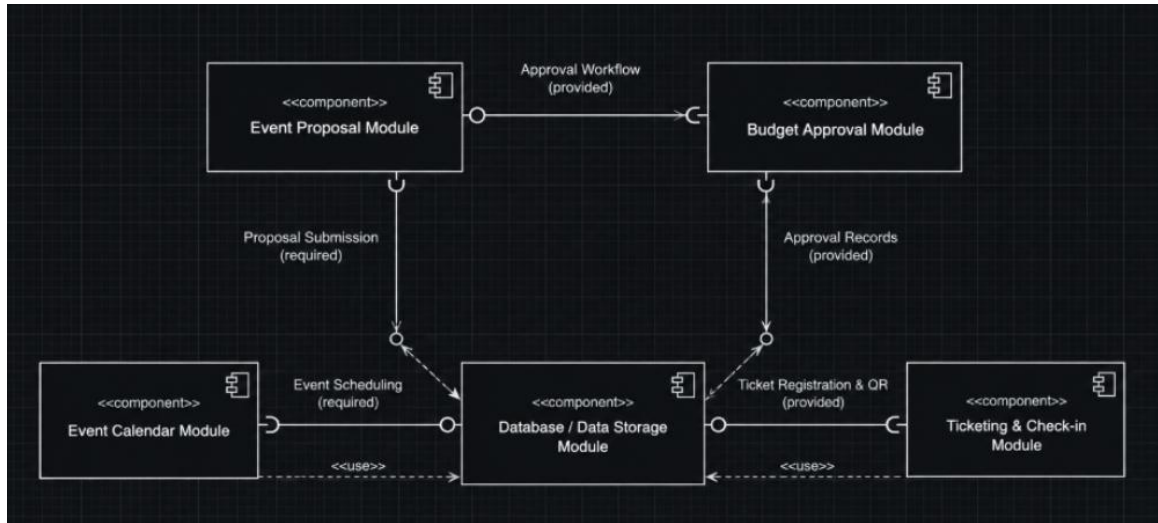
4. Component Identification

Identified Components

No.	Component	Responsibility
1	Event Proposal Module	Handles event proposal creation and submission by Club Leads
2	Budget Approval Module	Manages budget review and sequential approval by Finance Officer, Faculty Coordinator and Dean
3	Ticketing & Check-in Module	Handles attendee registration, ticket generation and QR-based check-in
4	Event Calendar Module	Manages scheduling and calendar display of approved events
5	Database / Data Storage Module	Provides centralized, persistent storage for proposals, approvals, registrations, tickets and event information

5. UML Component Diagram

Figure 1: Component Diagram of Student Club Event Ticketing & Budget Portal



The component diagram represents the major functional modules of the portal and their interactions. Provided interfaces (ball notation) represent services offered by a component, while required interfaces (socket notation) represent services needed from another component. Assembly connectors join compatible provided and required interfaces, while usage dependencies are shown as dashed arrows with the <<use>> stereotype, as seen between the Event Calendar Module, the Ticketing & Check-in Module, and the Database / Data Storage Module.

6. Interface and Dependency Description

Interface / Dependency	Components Involved	Purpose
Approval Workflow	Event Proposal ↔ Budget Approval	Transfers submitted event proposal information into the approval workflow
Proposal Submission	Event Proposal ↔ Data Storage	Stores submitted event proposal information
Approval Records	Budget Approval ↔ Data Storage	Stores approval and sign-off information
Event Scheduling	Event Calendar ↔ Data Storage	Stores and retrieves scheduled event information
Ticket Registration & QR	Ticketing & Check-in ↔ Data Storage	Stores registration and QR ticket information

These interfaces allow the modules to communicate while maintaining separation of responsibilities. The Event Proposal Module focuses on proposal management, the Budget Approval Module handles approval processing, the Ticketing & Check-in Module handles attendee access, and the Event Calendar Module manages scheduling. The Data Storage Module provides centralized persistence for the information shared by these modules, so every functional module depends on it while remaining independent of one another.

7. Architectural Justification

Architecture Selection

We chose Modular Monolith Architecture for the Student Club Event Ticketing & Budget Portal.

Architectural Choice

The system is designed as a modular monolith in which the Event Proposal, Budget Approval, Ticketing & Check-in, Event Calendar, and Data Storage functions are organised as separate, clearly bounded modules within a single deployable application.

Why Modular Monolith?

A Modular Monolith architecture was selected because the Student Club Event Ticketing & Budget Portal contains several closely related functions that need to communicate frequently. Event proposals, budget approvals, ticketing and check-in, and event scheduling are logically separate concerns, but they belong to the same overall application and share common data. A modular monolith provides clear separation between these functional areas without introducing the additional network and operational complexity associated with independently deployed microservices, while still keeping the system easier to develop, test and deploy than a fully distributed architecture.

Reason 1 - Closely Related Workflows

The system contains closely related workflows such as event proposal submission, sequential budget approval, attendee registration, ticket generation and event scheduling. A modular monolith allows these modules to communicate efficiently and share transactional data without introducing the network communication complexity of multiple independently deployed services.

Reason 2 - Manageable System Complexity

The portal is intended for student club activities and has a relatively manageable workload and user base. A modular monolith is therefore easier to develop, test, deploy and maintain than a microservices architecture, while still preserving clear separation between functional responsibilities.

Security Advantage

Role-Based Access Control (RBAC) can be enforced centrally across all modules within the single application. This ensures that users such as Club Leads, Faculty Coordinators, Finance Officers and Deans can access only the operations permitted for their role, and approval and sign-off information can be maintained centrally, improving control and traceability of sensitive budget decisions.

Performance Benefit

Because the modules operate within the same application and share the same runtime, communication between them does not require network calls between independently deployed

services. This reduces communication overhead and improves the responsiveness of frequent operations such as event registration and QR-based ticket validation during check-in.

8. Conclusion

The Student Club Event Ticketing & Budget Portal is best represented using a Modular Monolith architecture because its functional areas - event proposals, budget approval, ticketing and check-in, and event scheduling - are closely related but can still be cleanly separated into distinct modules. This architecture provides simpler development and deployment while supporting centralized security enforcement and efficient, low-latency communication between components. The UML Component Diagram demonstrates how these modules interact through provided and required interfaces and a shared data storage module.